dorodango

Draw squircles in Typst with tunable corner smoothing

Contents

Introduction . . . 2
API reference . . . 2
  squircle . . . 2
  superellipse . . . 5
  clothoid . . . 7
Concepts and examples . . . 10
  Squircle . . . 10
  Superellipse . . . 12
  Clothoid . . . 12
  Common parameters . . . 13

Introduction

`dorodango` provides three functions (`squircle`, `superellipse`, and `clothoid`) that mirror the built-in `rect` element, replacing abrupt circular corners with smooth curve transitions.

Note that each family is driven by different parameters (see the API reference), so the comparison above is not like-for-like. While `squircle` and `clothoid` corner blends can extend along adjacent edges as space allows, `superellipse` corners always stay strictly within their radius.

API reference

squircle

Draws a rectangle with smoothly rounded corners.

Use `squircle` like `rect` when you want softer, more continuous corner transitions. With `smoothing: 0%`, it has the same geometry as a rounded `rect`.

Parameters

```
squircle(
  width: auto length ratio relative,
  height: auto length ratio relative fraction,
  fill: none color gradient tiling,
  stroke: auto none length color gradient tiling stroke dictionary,
  radius: length ratio relative dictionary,
  inset: length ratio relative dictionary,
  outset: length ratio relative dictionary,
  smoothing: length ratio relative dictionary,
  preserve-smoothing: bool,
  per-edge-smoothing: bool,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The squircle's width, relative to its parent container.

Default: `auto`

height `auto` or `length` or `ratio` or `relative` or `fraction`

The squircle's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the squircle.

When setting a fill, the default stroke disappears. To create a squircle with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the squircle. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top`: The top stroke.
  - `right`: The right stroke.
  - `bottom`: The bottom stroke.
  - `left`: The left stroke.
  - `x`: The left and right stroke.
  - `y`: The top and bottom stroke.
  - `rest`: The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the squircle's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner radius.
  - `top-right`: The top-right corner radius.
  - `bottom-right`: The bottom-right corner radius.
  - `bottom-left`: The bottom-left corner radius.
  - `left`: The top-left and bottom-left corner radii.
  - `top`: The top-left and top-right corner radii.
  - `right`: The top-right and bottom-right corner radii.
  - `bottom`: The bottom-left and bottom-right corner radii.
  - `rest`: The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the squircle's content. See `box`'s `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the squircle's size without affecting the layout. See `box`'s `outset` parameter for more details.

Default: `0pt`

smoothing `length` or `ratio` or `relative` or `dictionary`

How strongly to smooth the squircle's corners. Smoothing replaces the ends of each circular corner arc with Bézier transitions whose curvature gradually changes between the straight edges and the arc. This can be:

- A relative length for uniform corner smoothing.
- A dictionary describing the smoothing for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner smoothing.
  - `top-right`: The top-right corner smoothing.
  - `bottom-right`: The bottom-right corner smoothing.
  - `bottom-left`: The bottom-left corner smoothing.
  - `left`: The top-left and bottom-left corner smoothing.
  - `top`: The top-left and top-right corner smoothing.
  - `right`: The top-right and bottom-right corner smoothing.
  - `bottom`: The bottom-left and bottom-right corner smoothing.
  - `rest`: The smoothing for all corners except those for which the dictionary explicitly sets smoothing.

At `0%`, the corner is a quarter circle matching a rounded `rect`. At `100%`, it is two Bézier transitions with no circular arc.

A corner's two edges normally share one available space, the smaller of the two. See `per-edge-smoothing` to change that.

Default: `60%`

preserve-smoothing `bool`

Whether to preserve the requested smoothing when it exceeds available edge space. This has no effect when smoothing already fits.

If `false`, smoothing scales down to fit. If `true`, both the requested radius and smoothing are retained by compressing the Bézier transitions.

Default: `false`

per-edge-smoothing `bool`

Whether each half of a corner can use all the space available on its edge when requested smoothing exceeds available room along one or both edges.

If `false`, each corner's two transition angles stay symmetric and share the tighter edge limit. If `true`, both halves smooth independently.

Default: `false`

..body `none` or `content` or `str` or `symbol`

The content to place into the squircle. Strings and symbols are converted to content.

When this is omitted, the squircle takes on a default size of at most `45pt` by `30pt`.

superellipse

Draws a rectangle with superellipse (Lamé curve) corners.

Use `superellipse` like `rect` when you want cubic approximations of Lamé curve corners parameterized by a power exponent. For exponents above 2, the ideal Lamé curve has zero curvature where it meets the straight edges. At `exponent: 2`, the corners are circular arcs matching a rounded `rect`. Higher exponents make the corners squarer.

Parameters

```
superellipse(
  width: auto length ratio relative,
  height: auto length ratio relative fraction,
  fill: none color gradient tiling,
  stroke: auto none length color gradient tiling stroke dictionary,
  radius: length ratio relative dictionary,
  inset: length ratio relative dictionary,
  outset: length ratio relative dictionary,
  exponent: int float,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The superellipse's width, relative to its parent container.

Default: `auto`

height `auto` or `length` or `ratio` or `relative` or `fraction`

The superellipse's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the superellipse.

When setting a fill, the default stroke disappears. To create a superellipse with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the superellipse. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top`: The top stroke.
  - `right`: The right stroke.
  - `bottom`: The bottom stroke.
  - `left`: The left stroke.
  - `x`: The left and right stroke.
  - `y`: The top and bottom stroke.
  - `rest`: The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the superellipse's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner radius.
  - `top-right`: The top-right corner radius.
  - `bottom-right`: The bottom-right corner radius.
  - `bottom-left`: The bottom-left corner radius.
  - `left`: The top-left and bottom-left corner radii.
  - `top`: The top-left and top-right corner radii.
  - `right`: The top-right and bottom-right corner radii.
  - `bottom`: The bottom-left and bottom-right corner radii.
  - `rest`: The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the superellipse's content. See `box`'s `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the superellipse's size without affecting the layout. See `box`'s `outset` parameter for more details.

Default: `0pt`

exponent `int` or `float`

The Lamé curve exponent n in $|\frac{x}{p}|^n + |\frac{y}{p}|^n = 1$.

Controls the corner shape profile. Values must be finite and are clamped into $[2, 12]$. Below 2 the curve would bulge inward, while above 12 the three-cubic fit degrades and control points can leave the corner's footprint. Near the upper bound the fit trades fidelity for containment. At 2 the curve is a circle, and the corner drawn is the circular arc `rect` draws rather than an approximation.

Default: `5`

..body `none` or `content` or `str` or `symbol`

The content to place into the superellipse. Strings and symbols are converted to content.

When this is omitted, the superellipse takes on a default size of at most `45pt` by `30pt`.

clothoid

Draws a rectangle with clothoid (Euler-spiral) blend corners.

Use `clothoid` like `rect` when you want cubic approximations of Euler-spiral corner transitions, whose ideal curvature ramps linearly along arc length. At `smoothing: 0%`, it has the same geometry as a rounded `rect`.

Parameters

```
clothoid(
  width: auto length ratio relative,
  height: auto length ratio relative fraction,
  fill: none color gradient tiling,
  stroke: auto none length color gradient tiling stroke dictionary,
  radius: length ratio relative dictionary,
  inset: length ratio relative dictionary,
  outset: length ratio relative dictionary,
  smoothing: length ratio relative dictionary,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The clothoid's width, relative to its parent container.

Default: `auto`

height `auto` or `length` or `ratio` or `relative` or `fraction`

The clothoid's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the clothoid.

When setting a fill, the default stroke disappears. To create a clothoid with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the clothoid. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top`: The top stroke.
  - `right`: The right stroke.
  - `bottom`: The bottom stroke.
  - `left`: The left stroke.
  - `x`: The left and right stroke.
  - `y`: The top and bottom stroke.
  - `rest`: The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the clothoid's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner radius.
  - `top-right`: The top-right corner radius.
  - `bottom-right`: The bottom-right corner radius.
  - `bottom-left`: The bottom-left corner radius.
  - `left`: The top-left and bottom-left corner radii.
  - `top`: The top-left and top-right corner radii.
  - `right`: The top-right and bottom-right corner radii.
  - `bottom`: The bottom-left and bottom-right corner radii.
  - `rest`: The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the clothoid's content. See `box`'s `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the clothoid's size without affecting the layout. See `box`'s `outset` parameter for more details.

Default: `0pt`

smoothing `length` or `ratio` or `relative` or `dictionary`

How strongly to smooth the clothoid's corners. Smoothing splits the 90-degree corner rotation into clothoid spiral transitions (where curvature ramps linearly) and a central circular arc. This can be:

- A relative length for uniform corner smoothing.
- A dictionary describing the smoothing for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner smoothing.
  - `top-right`: The top-right corner smoothing.
  - `bottom-right`: The bottom-right corner smoothing.
  - `bottom-left`: The bottom-left corner smoothing.
  - `left`: The top-left and bottom-left corner smoothing.
  - `top`: The top-left and top-right corner smoothing.
  - `right`: The top-right and bottom-right corner smoothing.
  - `bottom`: The bottom-left and bottom-right corner smoothing.
  - `rest`: The smoothing for all corners except those for which the dictionary explicitly sets smoothing.

At `0%`, the corner is a quarter circle matching a rounded `rect`. At `100%`, it is two clothoid transitions meeting with no circular arc.

When the natural blend does not fit the tighter adjacent-edge budget, its clothoid lengths and effective circular radius scale down together. This keeps the angular smoothing proportions unchanged.

Default: `60%`

..body `none` or `content` or `str` or `symbol`

The content to place into the clothoid. Strings and symbols are converted to content.

When this is omitted, the clothoid takes on a default size of at most `45pt` by `30pt`.

Concepts and examples

Squircle

`squircle` draws Figma's squircle corner, a circular arc with a cubic Bézier shoulder at either end. `smoothing` splits the corner's 90° turn between the shoulders and the arc, so at `0%` the corner is a plain quarter circle and at `100%` the arc vanishes and the two shoulders meet. `preserve-smoothing` and `per-edge-smoothing`, both specific to `squircle`, control what happens when the requested corner does not fit.

Smoothing

`smoothing` controls the gradual transition between edges and corners. In the example below, the shapes use `0%`, `60%`, and `100%` smoothing.

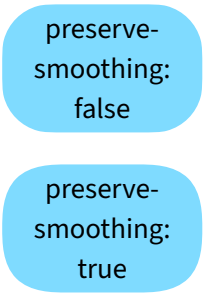
```typ
#grid(
  rows: 3,
  gutter: 12pt,
  ..(0%, 60%, 100%).map(s => align(center +
horizon)[
    #squircle(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      smoothing: s,
      fill: aqua,
    )[smoothing: #repr(s)]
  ]),
)
```

Preserve smoothing

Large radii and high smoothing both consume space along a corner's edges. For a corner with radius r and smoothing s, the required space along each edge is $p = (1 + s)r$. When p exceeds the available edge budget b, `preserve-smoothing` determines how the shape adjusts:

- `preserve-smoothing: false` (default) preserves the radius and reduces smoothing to $\frac{b}{r} - 1$.
- `preserve-smoothing: true` keeps both the requested radius and smoothing, compressing the Bézier transitions to fit the edge.

```typ
#grid(
  rows: 2,
  gutter: 12pt,
  ..(false, true).map(keep => align(center +
horizon)[
    #squircle(
      width: 75pt,
      height: 48pt,
      radius: 18pt,
      smoothing: 100%,
      preserve-smoothing: keep,
      fill: aqua,
    )[preserve-smoothing: #repr(keep)]
  ]),
)
```



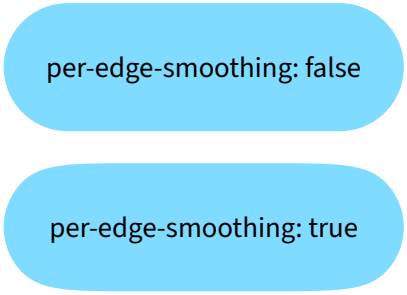
Per-edge smoothing

Each half of a corner uses space along one adjacent edge, and those edges may have different lengths. For example, on an elongated rectangle, a short edge might run out of space for smoothing while the long edge still has plenty of room.

- `per-edge-smoothing: false` (default) uses the tighter edge budget for both halves, keeping the corner symmetric.
- `per-edge-smoothing: true` allows each half to adapt to its own edge budget independently.

When `preserve-smoothing` is `false`, smoothing is reduced only on the tighter edge, giving the two halves different transition angles. When `preserve-smoothing` is `true`, both halves retain their transition angles, but the transition on the tighter edge is compressed to fit.

```
#grid(
  rows: 2,
  gutter: 12pt,
  ..(false, true).map(u => align(center +
horizon)[
    #squircle(
      width: 150pt,
      height: 48pt,
      radius: 25pt,
      smoothing: 100%,
      per-edge-smoothing: u,
      fill: aqua,
    )[per-edge-smoothing: #repr(u)]
  ]),
)
```



Superellipse

`superellipse` draws corners based on Lamé curves ($|\frac{x}{p}|^n + |\frac{y}{p}|^n = 1$), fitted with three cubic Bézier segments inside a square corner footprint of side p. For $n > 2$, the curve meets straight edges with zero curvature.

Exponent

The `exponent` parameter sets n, clamped to $[2, 12]$. At $n = 2$, the corner is an exact circular arc matching `rect`. Higher values make the corner squarer. Near $n = 12$, the cubic fit trades exact curve fidelity to stay strictly within the corner footprint.

```
#grid(
  rows: 3,
  gutter: 12pt,
  ..(2, 3, 6).map(n => align(center + horizon)[
    #superellipse(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      exponent: n,
      fill: aqua,
    )[exponent: #n]
  ]),
)
```

Clothoid

`clothoid` draws Euler-spiral corner blends, ramping curvature linearly from zero at the straight edge up to the curvature of a central circular arc. Each spiral transition is approximated by a single cubic Bézier segment.

Smoothing

`smoothing` splits the 90° corner turn between the spiral transitions and the central circular arc. At `0%`, the corner is a standard circular arc matching `rect`. At `100%`, the central arc vanishes and the two spirals meet directly.

When a blend requires more space than the adjacent edge allows, `clothoid` uniformly scales down both the spiral lengths and circular radius, keeping the angular smoothing proportions intact.

```
#grid(
  rows: 3,
  gutter: 12pt,
  ..(0%, 60%, 100%).map(s => align(center +
horizon)[
    #clothoid(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      smoothing: s,
      fill: aqua,
    )[smoothing: #repr(s)]
  ]),
)
```

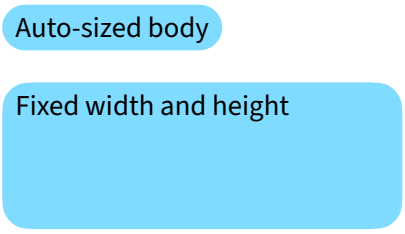
Common parameters

Although `squircle`, `superellipse`, and `clothoid` take different parameters to shape their corners, they share the common ones described below:

Size and body

`width` and `height` set the shape's layout dimensions. When both are `auto`, the shape sizes itself to fit the content passed in the body.

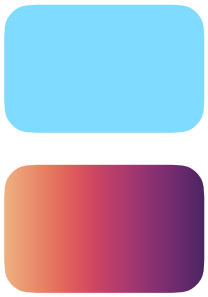
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(radius: 10pt, fill: aqua)[Auto-sized
body],
  squircle(
    width: 150pt,
    height: 55pt,
    radius: 10pt,
    fill: aqua,
  )[Fixed width and height],
)
```



Fill

`fill` sets the interior color, gradient, or tiling pattern.

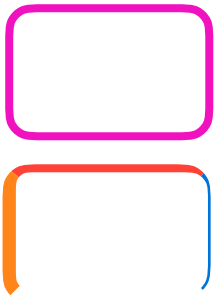
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(width: 75pt, height: 48pt, radius:
10pt, fill: aqua),
  squircle(width: 75pt, height: 48pt, radius:
10pt, fill: gradient.linear(..color.map.flare)),
)
```



Stroke

`stroke` sets the border outline, either uniformly or per side.

```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 10pt,
    stroke: 3pt + fuchsia,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 10pt,
    stroke: (top: 3pt + red, bottom: none, right:
blue, left: 5pt + orange),
  ),
)
```



Radius

`radius` controls corner rounding, either as a single value for all corners or per corner.

```
#grid(
  rows: 3,
  gutter: 12pt,
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 0pt,
    fill: aqua,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 35%,
    fill: aqua,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: (top-left: 4pt, rest: 20pt),
    fill: aqua,
  ),
)
```

Inset and outset

`inset` adds internal padding around the content, while `outset` extends the background drawing outward without affecting layout dimensions.

```
#grid(
  rows: 3,
  gutter: 12pt,
  squircle(
    width: 90pt,
    height: 48pt,
    radius: 10pt,
    fill: aqua,
  )[No inset or outset],
  squircle(
    width: 90pt,
    height: 48pt,
    inset: (x: 20pt, y: 10pt),
    radius: 10pt,
    fill: aqua,
  )[Inset adds padding],
  squircle(
    width: 90pt,
    height: 48pt,
    radius: 10pt,
    outset: 7pt,
    fill: aqua,
  )[Outset expands the drawing],
)
```